E-RAKSHAK HACKATHON 2026 · PROBLEM STATEMENT PS-08

Surat Traffic Intelligence

Data-Driven Traffic Optimization & Adaptive Infrastructure Intelligence

Alpha Devs — Rudraksh Fanse · Rushang Bagada · Ayushman Singh · Nikhil Pathak

A traffic measurement and signal-optimisation system built on Surat's existing CCTV network. It measures lane density and congestion from video, computes an adaptive signal plan from measured demand, detects sustained bottlenecks with the evidence that raised each one, and forecasts short-horizon congestion. It ships two signal controllers — and runs each only on data that can actually support it.

> **The one-line architecture.** Surat's junctions run Webster's method. Max-pressure is validated on the FLUID drone dataset. Neither borrows from the other, and an automated test fails if that ever stops being true.

1. Executive summary

Problem statement PS-08 asks for traffic density measurement, adaptive signal optimisation, bottleneck detection and infrastructure recommendations, using the CCTV network a city already has. We were given seven Surat clips — 1920×1080 at 20 fps, about 31.5 minutes in total — with no ground truth, no camera calibration and no signal timing data.

The system analyses all seven cameras into **2,451 two-second measurement windows** across **21 zones**, and produces:

Output	Result
Worst-approach degree of saturation	**1.01 → 0.929** — an over-capacity approach brought back under 1.0
Optimal cycle length	135.6 s, against a 120 s fixed-timer baseline
Mean delay	53.5 → 51.4 s/PCU (HCM control delay), –3.8%
How long the retiming lasts	**+8.2% demand headroom** before no timing can cope — about 1.6 years at 5% annual growth; after that the fix is physical
Bottleneck alerts	31, each with its triggering evidence and a specific recommendation
Zone forecasts	21, with prediction intervals
Max-pressure validation	Real fixed-time signals waste 41–66% of green; max-pressure 1.5–16%
Live control room	Every camera on one screen — replayed, or detected in real time — with alerts raised as their evidence completes and an acknowledge/resolve incident log

Two findings shaped the design more than any algorithm choice, and both are cases where the obvious implementation would have produced confident, plausible, wrong answers.

2. What the data actually supports

Reading the footage before writing the system changed the design twice.

2.1 Only two of seven cameras can support the requested measurement

Two cameras — GauravPath and Rokadiya Hanuman — watch divided, lane-marked carriageways. The other five watch a dense market street, a station forecourt, a recreational gate area and a bus depot platform. There is visibly no lane discipline in those scenes and no signal to optimise. "Lane density" is not a measurable quantity there, and forcing it would mean inventing numbers.

So the registry splits cameras into two modes. **Junction** cameras support per-lane density and signal-timing recommendations. **Area** cameras use functional zones — movement channel, encroachment, pedestrian area — and report occupancy and crowding instead. Every number the system reports stays defensible.

2.2 In the worst gridlock, detection counts fewer vehicles, not more

Camera	Detections	Actual condition
Rokadiya Hanuman	7,664	flowing arterial
Central Bus Depo	2,905	busy
GauravPath	1,957	free-flowing
Mahidharpura	**463**	**worst gridlock in the dataset**

At extreme density a wall of overlapping vehicles stops being separable and the detector's recall collapses. A system that equates more detections with more congestion would therefore rank Surat's worst street among its emptiest — confidently, and backwards, with every intermediate number looking entirely reasonable.

The fix. A texture-based occupancy index — Canny edge density inside each zone, normalised against absolute bounds measured across all seven cameras — plus an explicit saturation flag. It has no notion of an object, so occlusion cannot defeat it. The bounds must be absolute rather than per-zone: normalising a zone against its own observed range makes it its own yardstick and destroys any comparison between sites. Mahidharpura now flags 452 of 453 windows as saturated and ranks correctly as the most congested site.

3. Architecture

The pipeline has seven stages. Video frames are detected and tracked, located into lane or zone polygons, then measured along **two parallel paths** whose outputs feed signal optimisation, bottleneck detection and forecasting, and finally a two-track dashboard with a live control room.

Stage	Module	What it does

1. Detect & track	detector.py	YOLO11-S (UVH-26 fine-tune) + ByteTrack, 14 Indian vehicle classes
2. Locate	zones.py	Lane / zone polygons, anisotropic perspective scale model
3. Measure	metrics.py	Detection path (PCU density, speed, queue, flow) and occupancy path (texture index)
4. Optimise	signal_opt.py, maxpressure.py	Webster's cycle and splits; max-pressure with its observability gate
5. Detect bottlenecks	bottleneck_detect.py	Five sustained-pattern detectors with recommendations
6. Predict	forecast.py	Damped trend with prediction intervals
7. Present	api.py, live.py, web/	React control room: live monitor, incidents, map; two tracks, fifteen views

Which measurement path leads is decided per zone by a saturation check, and the active mode is carried in the output so the dashboard can be explicit about it. The saturation check uses two independent signatures — high density with heavy mutual overlap, and "full scene, empty count" where texture says packed but detection boxes cover almost nothing. Only the second catches the collapse case, because once detection fails there are too few boxes left to overlap.

4. Signal control — two controllers, one boundary

	Webster's	Max-pressure (Varaiya 2013)
Needs to observe	arrival flow on each approach	approach queue **and** the queue on the link it discharges into
Cameras required	one per approach	one per approach **plus one per receiving link**
Surat CCTV provides	exactly this	no camera watches a receiving link
Runs on	the 7 Surat clips — every operational figure	FLUID: 14 drone recordings of 3 real intersections
On Surat it reports	cycle, splits, delay, saturation	`greedy-queue` — and refuses to name a phase

Dropping the downstream term does not yield a degraded max-pressure. It yields **greedy queue control**, which Varaiya's §3.1 Example 2 proves fails to stabilise demands that max-pressure handles — on a junction as simple as 2×2. So rather than substitute zero and keep the better name, the implementation refuses: every movement carries whether its receiving-link queue was actually observed, and the decision object returns the greedy answer in separate fields so a caller cannot render it as max-pressure.

To validate the controller for real, we obtained a dataset that contains the missing term. FLUID is 14 drone recordings of three real signalised intersections, where the whole junction fits in one aerial frame, so every approach *and* every receiving link is visible at once.

5. Results

5.1 Surat — the operational system

The existing fixed timer runs a 120 s cycle with equal green splits, which drives the busier approach to a degree of saturation of **1.01** — over capacity, where the queue never clears within the cycle. Webster's optimal plan brings it to **0.929**.

The saturation crossing is the claim that survives any delay model. Delay itself falls 3.8% (53.5 → 51.4 s/PCU under HCM control delay), and we quote that smaller figure rather than the more flattering one available under other assumptions.

Two independent checks confirm the optimiser. Sweeping cycle length puts the empirical delay minimum at **130 s** against Webster's closed form of **135.6 s**. And the optimal plan equalises degree of saturation across approaches (0.929 / 0.929), which is the known optimality condition for Webster green allocation.

#	Site	Mode	Mean congestion	Saturated windows
1	Mahidharpura – Diamond Mkt	area	0.939	452 / 453
2	Gopi Talav Gate	area	0.749	167 / 456
3	Central Bus Depo	area	0.483	213 / 450
4	Rokadiya Hanuman	junction	0.437	0 / 244
5	Chauta Bazaar	area	0.205	4 / 453
6	Rly Station	area	0.099	45 / 302
7	GauravPath	junction	0.073	0 / 93

Plus **31 evidence-backed bottleneck alerts** and **21 zone forecasts** with prediction intervals.

How long the fix lasts. Webster's closed form also gives the reserve capacity: at the longest acceptable cycle (150 s) the junction stays serviceable while the total flow ratio is below 0.947. Measured it is **0.875** — **+8.2% headroom**, about **1.6 years** at 5% annual growth, and already past the 0.90 practical limit. Losing 10% of saturation flow on one approach (a parked bus near the stop line) pushes the best possible plan to x = 0.98. Beyond those points the remedy is road space or demand management, not retiming — the kind of answer an infrastructure recommendation needs.

5.2 Live operation — the control room

The dashboard opens on a **live monitor**: all seven cameras on a street map coloured by level of service, the focused camera's video, live zone states, the alerts in force and the zones forecast to worsen within a minute. Recorded windows replay on a shared clock as if arriving now, and each alert fires the moment its evidence completes — quoting only the windows up to that instant; the verification suite checks that one lap raises exactly the 31 recorded alerts and none early. One click runs the real pipeline on a camera in real time, skipping frames rather than falling behind, and watches each track for a vehicle standing still in a flowing lane. Alerts become work in an incident log — acknowledged, resolved, with notes and names — and a printable briefing summarises the network for a meeting.

5.3 FLUID — the max-pressure validation

Intersection	Cycle	Real signal: green with nothing to relieve	Max-pressure
FI (4-way, 3-phase)	96 s	**41.1%**	1.5%
FIDRT (dedicated right turns)	123 s	**66.3%**	2.3%
TI (T-intersection)	71 s	**46.6%**	16.0%

These real fixed-time plans spend 41–66% of their green on a phase whose queue had already cleared, or whose receiving link was fuller than the approach feeding it.

That is deliberately **not** a delay-reduction claim. It is an observation about the recorded signals, made from the recorded queues — had max-pressure actually run, the queues would have evolved differently. The counterfactual needs a model, so we built a point-queue simulator over two junctions in series, with finite receiving-link storage so spillback exists.

Scenario	Fixed-time	Max-pressure	Spillback	Verdict
Steady, balanced	**18.6 s/PCU**	20.8 s/PCU	0 → 0 s	fixed-time wins by 12%
Demand swings between approaches	24.5 s/PCU	**18.5 s/PCU**	0 → 0 s	max-pressure –25%
Downstream blockage	150.0 s/PCU	**102.9 s/PCU**	289 → 88 s	max-pressure –31%

The operator's decision rule — narrower than the literature's enthusiasm suggests. Max-pressure earns its place when the *split* of demand moves between approaches, and above all when something downstream backs up. On steady demand it is the **worse** choice, because every extra switch pays an inter-green for a benefit that is not there. A system offering it as a straight upgrade would be selling the wrong thing. Ours measures which regime a junction is in and recommends accordingly.

6. Innovation and unique value

6.1 The observability gate

Max-pressure declines to name a phase when its defining term is unmeasurable, and returns the weaker controller's answer under its own name. The refusal is structural — it lives in the shape of the returned object, not in a warning string a caller can ignore.

6.2 Occlusion-proof measurement

The texture-based occupancy index keeps working exactly where object detection collapses. This is the difference between a plausible ranking and a correct one.

6.3 Measurement separated from estimation

No camera here has a verified metric scale — three independent attempts to recover it disagreed with each other by a factor of two to four. So the system separates what it **measures** from what it **estimates**. Congestion index, level of service, site ranking, occupancy and the entire signal-timing

comparison are scale-free and trustworthy. Anything in km/h or PCU/km/lane is displayed with a `~` prefix behind a persistent banner. **Every headline result above is in the first category.**

6.4 Documentation that cannot drift

The verification suite recomputes every figure quoted in the README and engineering report and fails if a number changes in the code but not the documentation, or the reverse.

7. Verification

`python tests/verify_system.py` runs 21 end-to-end checks, `tests/test_api.py` 50 API checks, and `tests/e2e_pipeline.py` drives every detector flow — uploads, calibration, live detection — through the real API. CI runs the first two on every push. Beyond recomputing the documented figures, the suite asserts the things most likely to break silently:

- **The separation** — the Surat plan is recomputed under both controller settings and must come out byte-identical, and the string `maxpressure` must not appear in any of the nine measurement and analysis modules.
- **The observability gate** — max-pressure must *refuse* on Surat and *run* on FLUID.
- **Scale invariance** — multiplying every queue by 7.3 must leave the phase choice unchanged, and scale the pressure by exactly 7.30.
- **Simulator fairness** — both controllers must be charged the same lost time, and all three demand scenarios must genuinely differ.
- **Geometry** — the world-to-pixel fit on every FLUID site must land within 5 px.
- **Physical sanity** — no zero-length measurement window, and no speed, density or occupancy outside its physical range.
- **Live replay is honest** — one lap raises exactly the recorded alerts, each no earlier than its condition began.
- **Headroom agrees with the optimiser** — scaling demand by the reported reserve puts the best plan at x = 1.000.

8. Challenges faced

Eleven bugs were found and fixed during development. Four are worth showing, because three of the four produced numbers that looked entirely reasonable on screen.

Symptom	Root cause	How it was caught
Speeds 5× too low — 4–8 km/h on a free-flowing arterial	Isotropic pixel-to-metre scale; vertical and horizontal pixels treated as covering equal ground	Physical implausibility of the reported speed
Flow 10× too high — 12,000–23,000 PCU/hr	Counting new ByteTrack IDs as arrivals; IDs churn constantly in dense mixed traffic	Compared against real saturation flow ≈1,800 PCU/hr

The "optimised" plan was worse than the baseline (−9.5%)	Flow ratio computed on the critical lane but saturation on the whole approach — mixed units suppressed the overflow-delay term	Swept cycle length and found the delay curve had no minimum, which is impossible if the implementation is correct
Max-pressure named a phase it had no data for	A missing downstream queue silently treated as zero	Audit against Varaiya §3.1 — the result was a different, provably weaker controller

What caught these was checking outputs against physical limits and known theory, not testing that the code ran.

9. Honest limitations

- **The metric scale is not calibrated.** Converting pixels to metres needs one true ground reference, and this dataset ships none. One measured road width promotes every metric figure to primary, and the Calibration view does it in about two minutes without a site visit.
- **Lane polygons are traced and heatmap-validated, not surveyed.**
- **The two junction cameras face different corridors**, not opposing approaches of one intersection. The combined signal plan therefore demonstrates the method on real measured demand rather than being deployable at a specific junction — a configuration change, not a code change.
- **Forecasting has minutes of history, not weeks.**
- **FLUID is Chinese traffic.** It validates that the controller implementation is correct on data containing the terms it needs. No claim from it transfers to Surat's fleet, and the dashboard keeps the two datasets on tracks that cannot render each other's data.

10. Future scope

Immediate — no new hardware

- **One surveyed road width** promotes every metric figure from estimate to primary.
- **Cameras on receiving links** unlock true max-pressure on Surat; the controller is already implemented and validated.
- **A live camera feed.** The live monitor already runs the full pipeline in real time on the recorded clips; reading a camera's RTSP stream is the remaining piece.

Longer term

- **Seasonal forecasting** once weeks of continuous operation justify it.
- **Coordinated corridors** — green-wave offsets across adjacent junctions; the simulator already models two in series.
- **City scale** — a message bus and a spatial database for hundreds of cameras, with the measurement core and the React control room unchanged.

11. Technology and attribution

Layer	Choice	Licence / note
Detector weights	UVH-26-MV-YOLOv11-S	AGPL-3.0 weights, CC-BY-4.0 dataset. Used as published — we did not train it.
Detection / tracking	Ultralytics YOLO11 + ByteTrack	AGPL-3.0
Validation dataset	FLUID	CC BY 4.0, DOI 10.6084/m9.figshare.29974954
Density unit	PCU per IRC:106-1990	Standard implemented, not imported
Saturation flow	IRC:SP:41-1994, S = 525·W	Standard implemented
Pedestrian minimum green	IRC:103-2012	Standard implemented
Signal methods	Webster (1958), HCM control delay, Varaiya (2013)	Published methods implemented
Dashboard	React 19 + FastAPI, Plotly, Leaflet / OpenStreetMap	MIT / BSD-2 / ODbL map data

All 34 Python modules — roughly 17,500 lines — and the React front end were written for this hackathon. The project is released under AGPL-3.0, inherited from Ultralytics. Full detail, including what is ours versus third-party, is in `ATTRIBUTION.md`.

12. Running the system

The metrics CSVs are committed, so the entire Webster track works immediately — no video download, no GPU, no model weights:

```
pip install -r requirements.txt
cd web && npm install && npm run build && cd ..
python src/api.py
```
→ http://127.0.0.1:8000

The full pipeline from raw video uses `requirements-full.txt` and adds detection, tracking and the interactive calibration tool. Deployment guidance — including what a hosted instance can and cannot show — is in `DEPLOYMENT.md`; the measurement data contract is in `DATA_SCHEMA.md`; and the complete engineering record, with every design decision and every bug, is in `REPORT.md`.